IP Addressing
from Binary to Subnetting

A complete, detailed course: binary & hexadecimal, IPv4 structure, classes,
public & private addresses, masks & CIDR, subnetting, VLSM, supernetting, NAT and IPv6

Level: beginner to intermediate · Edition: November 2026

Table of contents

1. **Binary numbers** – the foundation of everything
2. Hexadecimal
3. The binary AND operation
4. Structure of an IPv4 address
5. Address classes (classful addressing)
6. Public, private and special addresses
7. Unicast, broadcast, multicast, anycast
8. The subnet mask and CIDR notation
9. Network address, broadcast address and host range
10. **Subnetting** (fixed-length, FLSM)
11. Fast subnetting: the “magic number” method
12. **VLSM** – Variable Length Subnet Masking
13. **Supernetting** and route summarization
14. Wildcard masks
15. NAT and PAT
16. IPv6 addressing
17. Reference tables
18. Exercises with detailed answers

> **How to use this course**
> Do not skip chapter 1. Subnetting is only “difficult” for people who do not yet read binary comfortably; once powers of two are automatic, every subnetting problem becomes simple arithmetic. Do every small exercise with pencil and paper before checking the answer.
>
> Colour code used in binary examples: **network bits**, **subnet (borrowed) bits**, **host bits**.

1. Binary numbers

1.1 Why binary?

Computers store everything as electrical states: on or off. Each state is a **bit** (binary digit) that is either **0** or **1**. A group of 8 bits is a **byte**, also called an **octet** in networking. An IPv4 address is simply 32 bits; we write it as four decimal numbers only because humans read decimal more easily.

1.2 Positional number systems

In decimal (base 10) each position is worth ten times the position to its right:

Position	$10^3 = 1000$	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
4 725	4	7	2	5

4725 = 4×1000 + 7×100 + 2×10 + 5×1. Binary (base 2) works **exactly** the same way, but each position is worth **twice** the one to its right, and the only digits are 0 and 1.

1.3 The 8 bit values of an octet – learn them by heart

Bit position	7	6	5	4	3	2	1	0
Power of 2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	**128**	**64**	**32**	**16**	**8**	**4**	**2**	**1**

- The left-most bit is the **most significant bit** (MSB, worth 128); the right-most is the **least significant bit** (LSB, worth 1).
- The largest value of an octet is 11111111 = 128+64+32+16+8+4+2+1 = **255**; the smallest is 00000000 = **0**. That is why each part of an IPv4 address is between 0 and 255: there are 2^8 = **256** possible values.
- With *n* bits you can write $\mathbf{2^n}$ different values (from 0 to $2^n - 1$). This single rule is the heart of subnetting.

1.4 Binary → decimal

Write the values 128 64 32 16 8 4 2 1 above the bits and **add the values where there is a 1**.

Value	128	64	32	16	8	4	2	1	Total
11010110	1	1	0	1	0	1	1	0	128+64+16+4+2 = **214**
10101000	1	0	1	0	1	0	0	0	128+32+8 = **168**
00001010	0	0	0	0	1	0	1	0	8+2 = **10**
01111111	0	1	1	1	1	1	1	1	64+32+16+8+4+2+1 = **127**

Shortcut

A number made of only ones is $2^n - 1$: 1111 = 15, 111111 = 63, 01111111 = 127. Thus 01111111 = 128 − 1.

1.5 Decimal → binary

Method 1 – subtraction (the fastest for IP addresses)

Go through 128, 64, 32, 16, 8, 4, 2, 1. If the value fits in what remains, write **1** and subtract it; otherwise write **0**.

Example: convert **172**.

Value	Does it fit?	Bit	Remainder
128	172 ≥ 128 yes	1	172 − 128 = 44
64	44 ≥ 64 no	0	44
32	44 ≥ 32 yes	1	12
16	12 ≥ 16 no	0	12
8	12 ≥ 8 yes	1	4
4	4 ≥ 4 yes	1	0
2	no	0	0
1	no	0	0

Result: 172 = **10101100**.

Method 2 – successive division by 2

Divide by 2 repeatedly and note the remainders; read them **from bottom to top**.

Division	Quotient	Remainder
172 ÷ 2	86	0
86 ÷ 2	43	0
43 ÷ 2	21	1
21 ÷ 2	10	1
10 ÷ 2	5	0
5 ÷ 2	2	1
2 ÷ 2	1	0
1 ÷ 2	0	1 ↑ read upward

Reading upward: **10101100** – the same result. Always pad to 8 bits with zeros on the left: 10 = 1010 → **00001010**.

1.6 The mask values: bits set from the left

Subnet masks are built by turning bits on **from the left**. Only nine values are therefore possible in an octet of a mask. Memorise this table — you will use it constantly.

Bits set to 1	Binary	Decimal	Block size (256 − value)
0	00000000	0	256
1	10000000	128	128
2	11000000	192	64
3	11100000	224	32
4	11110000	240	16
5	11111000	248	8
6	11111100	252	4
7	11111110	254	2
8	11111111	255	1

Each line adds the next bit value: 128, 128+64 = 192, 192+32 = 224, 224+16 = 240, and so on.

1.7 Powers of two

Subnetting questions are always “how many bits do I need for *x* things?” – i.e. find the smallest *n* with $2^n \geq x$.

Power	Value	Power	Value
2^0	1	2^{12}	4,096
2^1	2	2^{13}	8,192
2^2	4	2^{14}	16,384
2^3	8	2^{15}	32,768
2^4	16	2^{16}	65,536
2^5	32	2^{17}	131,072
2^6	64	2^{18}	262,144
2^7	128	2^{19}	524,288
2^8	256	2^{20}	1,048,576
2^9	512	2^{21}	2,097,152
2^{10}	1,024	2^{22}	4,194,304
2^{11}	2,048	2^{23}	8,388,608
…		2^{32}	4,294,967,296

Practice before going further

Convert to binary: 255, 128, 192, 224, 10, 100, 33. Convert to decimal: 11110000, 00011111, 10000001, 01000000. (Answers: 11111111, 10000000, 11000000, 11100000, 00001010, 01100100, 00100001 / 240, 31, 129, 64.)

2. Hexadecimal

Hexadecimal (base 16) uses 16 digits: 0–9 then A=10, B=11, C=12, D=13, E=14, F=15. Its great advantage: **one hex digit = exactly 4 bits** (a *nibble*), so conversion with binary is immediate. Hex is used for MAC addresses and **IPv6**.

Dec	Hex	Binary	Dec	Hex	Binary
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	10	A	1010
3	3	0011	11	B	1011
4	4	0100	12	C	1100
5	5	0101	13	D	1101
6	6	0110	14	E	1110
7	7	0111	15	F	1111

Example: 172 = 1010 1100 → A C → **0xAC**. Back to decimal: A×16 + C = 10×16 + 12 = 172. Another: 0x3E = 0011 1110 = 3×16 + 14 = 62.

3. The binary AND operation

A device finds the network an address belongs to by computing **address AND mask**, bit by bit. The rule is: the result is 1 **only if both bits are 1**.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Consequence: where the mask has 1s, the address bit is **kept**; where the mask has 0s, the bit becomes **0**. ANDing with 255 keeps an octet unchanged, ANDing with 0 erases it.

Example: in which network is 192.168.10.77 with mask 255.255.255.192 (/26)?

```
Address  192.168.10.77    11000000.10101000.00001010.01001101
Mask     255.255.255.192  11111111.11111111.11111111.11000000
         ------------------------------------------------------ AND
Network  192.168.10.64    11000000.10101000.00001010.01000000
```

Only the last octet needs work: 77 = 01001101, AND 11000000 → 01000000 = **64**. The network is **192.168.10.64**.

> **Why it matters**
> When a host sends a packet, it ANDs its own address and the destination address with its mask. Same result → the destination is **local**, the packet is delivered directly (ARP). Different result → the destination is **remote**, the packet is sent to the **default gateway**.

4. Structure of an IPv4 address

An IPv4 address (Internet Protocol version 4, RFC 791) is a **32-bit** number. It is written in **dotted-decimal notation**: four octets converted to decimal and separated by dots.

```
Decimal :      192   .    168   .     1    .    25
Binary  :  11000000 . 10101000 . 00000001 . 00011001
           |<------------------- 32 bits ------------------->|
```

- Total address space: 2^{32} = **4,294,967,296** addresses.
- Each address has two parts: the **network portion** (identical for all devices of the same network, like a street name) and the **host portion** (unique for each device in it, like a house number).
- The boundary between the two parts is given by the **subnet mask** (chapter 8).

```
192.168.1.25 /24     11000000.10101000.00000001.00011001
                     |<------ network: 24 bits ------>|<host: 8>|
```

Invalid addresses

`192.168.1.256` or `10.300.0.1` do not exist: an octet cannot exceed 255. Leading zeros (`192.168.001.010`) should be avoided – some software reads them as octal.

5. Address classes (classful addressing)

From 1981 to 1993 the address space was divided into fixed **classes**. The class is identified by the **first bits** of the first octet. Classes are obsolete for routing (replaced by CIDR in 1993), but they are still used in vocabulary, in exams, and to define default masks.

Class	First bits	First octet range	Default mask	Networks	Hosts per network	Use
A	0xxxxxxx	1 – 126	255.0.0.0 (/8)	126	16,777,214	Very large networks
B	10xxxxxx	128 – 191	255.255.0.0 (/16)	16,384	65,534	Medium networks
C	110xxxxx	192 – 223	255.255.255.0 (/24)	2,097,152	254	Small networks
D	1110xxxx	224 – 239	—	Multicast groups		Multicast
E	1111xxxx	240 – 255	—	Reserved		Experimental

Network 0 and network 127 are excluded from class A (0 = “this network”, 127 = loopback), hence 126 usable class A networks. Class B: 2^{14} networks (16 bits minus the 2 fixed bits); class C: 2^{21} networks.

How to derive the ranges: class B starts with `10`, so the first octet goes from 10000000 (128) to 10111111 (191). Class C starts with `110`: 11000000 (192) to 11011111 (223).

> **Why classes were abandoned**
> A company needing 2,000 addresses was too big for a class C (254) and received a class B (65,534): 63,000 addresses wasted. This waste, and the explosion of routing tables, led to **CIDR** (Classless Inter-Domain Routing, RFC 1519, 1993): the mask can now have **any length**. Today, “classful” only means the default mask.

6. Public, private and special addresses

6.1 Public addresses

A **public** address is globally unique and routable on the Internet. Distribution is hierarchical:



IANA allocated its last free /8 blocks in 2011; all RIRs have now exhausted or nearly exhausted their IPv4 pools. This **IPv4 exhaustion** is why we use private addresses with NAT (chapter 15), why ISPs use Carrier-Grade NAT, and why IPv6 exists (chapter 16). IPv4 blocks are now bought and sold on a transfer market.

6.2 Private addresses (RFC 1918)

Three ranges are reserved for internal use. Anyone can use them freely, in any number of organisations at the same time, but they are **never routed on the Internet**: to reach the Internet, they must be translated into a public address by **NAT**.

Block	Range	Number of addresses	Classful equivalent
10.0.0.0/8	10.0.0.0 – 10.255.255.255	16,777,216	1 class A
172.16.0.0/12	172.16.0.0 – 172.31.255.255	1,048,576	16 class B
192.168.0.0/16	192.168.0.0 – 192.168.255.255	65,536	256 class C

Classic trap: 172.16.0.0/12
The /12 covers 172.**16** to 172.**31** only. Why? 16 = 0001**0000** and 31 = 0001**1111**: the first 4 bits of the second octet (0001) are fixed by the /12 (8 + 4 = 12). So **172.15.x.x** and **172.32.x.x** are **public** addresses.

6.3 Special-purpose addresses

Block	Name	Meaning
0.0.0.0/8	“This network”	0.0.0.0 = no address yet (DHCP client source) or “all addresses” (default route 0.0.0.0/0, server listening on all interfaces).
127.0.0.0/8	Loopback	The host itself; 127.0.0.1 = *localhost*. Packets never leave the machine.
169.254.0.0/16	Link-local (APIPA)	Self-assigned when DHCP fails. Not routable.
100.64.0.0/10	Shared address space	Carrier-Grade NAT between ISP and customer routers (RFC 6598).
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	TEST-NET-1/2/3	Documentation and examples only (RFC 5737).
198.18.0.0/15	Benchmarking	Network equipment performance tests.
192.88.99.0/24	Deprecated	Formerly 6to4 relay anycast.
224.0.0.0/4	Multicast	Former class D (e.g. 224.0.0.5 OSPF, 224.0.0.251 mDNS).
240.0.0.0/4	Reserved	Former class E.
255.255.255.255	Limited broadcast	All hosts of the local network; never forwarded by routers.

In each subnet, two addresses are also reserved: the **network address** (all host bits at 0) and the **broadcast address** (all host bits at 1) – see chapter 9.

7. Unicast, broadcast, multicast, anycast

Type	Recipients	Example
Unicast	One single interface	192.168.1.25 – normal traffic
Broadcast	All hosts of a network (IPv4 only)	192.168.1.255 (directed), 255.255.255.255 (limited) – ARP, DHCP Discover
Multicast	Hosts that joined a group	224.0.0.0/4 – video streaming, routing protocols (OSPF 224.0.0.5)
Anycast	The nearest of several hosts sharing the same address	8.8.8.8 and 1.1.1.1 are announced from many data centres; DNS root servers

A **broadcast domain** is the set of hosts reached by a broadcast; routers stop broadcasts. Creating subnets (and VLANs) splits a big broadcast domain into smaller ones – one of the main reasons for subnetting.

8. The subnet mask and CIDR notation

8.1 The subnet mask

The mask is a 32-bit value with **1s for the network part and 0s for the host part**. The 1s are always contiguous and on the left.

```
255.255.255.0     11111111.11111111.11111111.00000000   → 24 network bits, 8 host bits
255.255.240.0     11111111.11111111.11110000.00000000   → 20 network bits, 12 host bits
255.255.255.252   11111111.11111111.11111111.11111100   → 30 network bits, 2 host bits
```

`255.255.0.255` or `255.255.255.64` are **invalid masks**: the 1s are not contiguous (64 = 01000000 is not in the table of section 1.6).

8.2 CIDR (prefix) notation

Instead of writing the mask, we write a slash followed by the **number of 1 bits**: 192.168.1.25 **/24** = mask 255.255.255.0. This number is called the **prefix length**.

Converting between the two

- **/20 → mask**: 20 = 8 + 8 + 4 → 255 . 255 . (4 bits = 240) . 0 → **255.255.240.0**.
- **255.255.255.224 → prefix**: 8 + 8 + 8 + (224 = 3 bits) = **/27**.
- **/13 → mask**: 8 + 5 → 255 . (5 bits = 248) . 0 . 0 → **255.248.0.0**.

8.3 The two fundamental formulas

> With **h** host bits (h = 32 − prefix):
> - Number of addresses in the subnet = $\mathbf{2^h}$
> - Number of usable hosts = $\mathbf{2^h - 2}$ (minus the network address and the broadcast address)
>
> When **s** bits are borrowed from the host part to create subnets:
> - Number of subnets = $\mathbf{2^s}$

Examples: /24 → h = 8 → 256 addresses, 254 hosts. /26 → h = 6 → 64 addresses, 62 hosts. /30 → h = 2 → 4 addresses, 2 hosts (ideal for a link between two routers). /31 → 2 addresses, both usable on point-to-point links (RFC 3021). /32 → one single host (loopback interfaces, host routes).

The complete CIDR table is in chapter 17.

9. Network address, broadcast address and host range

For any address with its mask, you can compute four values:

Value	Rule in binary
Network address (subnet ID)	All host bits set to **0** (= address AND mask)
Broadcast address	All host bits set to **1**
First usable host	Network address + 1
Last usable host	Broadcast address − 1

Worked example 1: 192.168.10.77/26

```
Address    192.168.10.77    11000000.10101000.00001010.01001101
Mask /26   255.255.255.192  11111111.11111111.11111111.11000000
Network    192.168.10.64    11000000.10101000.00001010.01000000   host bits → 0
Broadcast  192.168.10.127   11000000.10101000.00001010.01111111   host bits → 1
```

Network **192.168.10.64**, first host **192.168.10.65**, last host **192.168.10.126**, broadcast **192.168.10.127**, $2^6 - 2 =$ **62** hosts.

Worked example 2: 172.16.45.200/20

/20 = 255.255.240.0. The first two octets are entirely network, the fourth entirely host. Only the 3rd octet is shared (“interesting octet”): 4 network bits + 4 host bits.

```
Address  10101100.00010000.00101101.11001000
3rd octet: 45 = 00101101   → host bits to 0: 00100000 = 32  ;  host bits to 1: 00101111 = 47
```

Network **172.16.32.0**, broadcast **172.16.47.255** (the 4th octet is all host bits → 0 for the network, 255 for the broadcast), first host **172.16.32.1**, last host **172.16.47.254**, hosts = $2^{12} - 2 =$ **4,094**.

Worked example 3: 10.45.123.77/13

/13 = 255.248.0.0. Interesting octet: the 2nd (5 network bits, 3 host bits).

```
Address  00001010.00101101.01111011.01001101
2nd octet: 45 = 00101101   → network: 00101000 = 40   broadcast: 00101111 = 47
```

Network **10.40.0.0**, broadcast **10.47.255.255**, hosts 10.40.0.1 – 10.47.255.254, $2^{19} - 2 =$ **524,286** hosts.

An address ending in .0 or .255 is not always invalid

In 192.168.0.0/23 (192.168.0.0 – 192.168.1.255), the address 192.168.0.255 and 192.168.1.0 are **normal host addresses**. Only the first and last addresses of the whole block are reserved. Always calculate; never judge by the last octet alone.

10. Subnetting (FLSM)

10.1 What and why

Subnetting means dividing one network into several smaller networks by **borrowing bits from the host part** and adding them to the network part. Reasons:

- Reduce the size of **broadcast domains** (less broadcast traffic, better performance).
- **Security**: separate departments, servers, guests, IoT, and filter traffic between them on the router/firewall.
- Use the address space **efficiently** and match the physical/logical organisation (buildings, floors, VLANs).
- Easier administration and troubleshooting.

In **FLSM** (Fixed Length Subnet Mask) all subnets have the same mask.

```
Before : 192.168.1.0 /24   11000000.10101000.00000001.00000000
After  : /26               11000000.10101000.00000001.00000000
                                      network (24)          subnet (2) host (6)
```

10.2 The method, step by step

1. **Determine how many bits to borrow**: smallest s with $2^s \geq$ number of subnets needed – *or* smallest h with $2^h - 2 \geq$ hosts needed per subnet (then borrow the remaining bits).
2. **New prefix** = old prefix + s. Write the new mask.
3. **Block size** (increment) = 256 − value of the mask in the interesting octet (= $2^{\text{number of host bits in that octet}}$).
4. **List the subnets**: start at the original network and add the block size each time.
5. For each subnet: first host = network + 1, broadcast = next network − 1, last host = broadcast − 1.

10.3 Example A – by number of subnets: 192.168.1.0/24 into 4 subnets

1. 4 subnets → $2^2 = 4$ → borrow **s = 2** bits.
2. New prefix /24 + 2 = **/26** → mask 255.255.255.**192**.
3. Block size = 256 − 192 = **64**.
4. Subnets: .0, .64, .128, .192.
5. Hosts per subnet: $2^6 - 2 = 62$.

#	Subnet	First host	Last host	Broadcast	Hosts
1	192.168.1.0/26	192.168.1.1	192.168.1.62	192.168.1.63	62
2	192.168.1.64/26	192.168.1.65	192.168.1.126	192.168.1.127	62
3	192.168.1.128/26	192.168.1.129	192.168.1.190	192.168.1.191	62
4	192.168.1.192/26	192.168.1.193	192.168.1.254	192.168.1.255	62

In binary, the two borrowed bits take every combination 00, 01, 10, 11 → 0, 64, 128, 192.

10.4 Example B – by number of hosts: 50 hosts per subnet in 192.168.1.0/24

$2^5 - 2 = 30$ is too small; $2^6 - 2 = 62 \geq 50$ → keep **h = 6** host bits → prefix $32 - 6 =$ **/26**. Same result as example A: 4 subnets of 62 hosts.

10.5 Example C – 192.168.100.0/24 into 8 subnets

$8 = 2^3$ → borrow 3 bits → **/27** (255.255.255.224), block 32, 30 hosts each.

#	Subnet	First host	Last host	Broadcast	Hosts
1	192.168.100.0/27	192.168.100.1	192.168.100.30	192.168.100.31	30
2	192.168.100.32/27	192.168.100.33	192.168.100.62	192.168.100.63	30
3	192.168.100.64/27	192.168.100.65	192.168.100.94	192.168.100.95	30
4	192.168.100.96/27	192.168.100.97	192.168.100.126	192.168.100.127	30
5	192.168.100.128/27	192.168.100.129	192.168.100.158	192.168.100.159	30
6	192.168.100.160/27	192.168.100.161	192.168.100.190	192.168.100.191	30
7	192.168.100.192/27	192.168.100.193	192.168.100.222	192.168.100.223	30
8	192.168.100.224/27	192.168.100.225	192.168.100.254	192.168.100.255	30

10.6 Example D – class B: 172.16.0.0/16, 500 hosts per subnet

$2^9 - 2 = 510 \geq 500$ → h = 9 → prefix **/23** → mask 255.255.**254**.0. Borrowed bits: $23 - 16 = 7$ → $2^7 =$ **128 subnets**. The interesting octet is the 3rd, block size $256 - 254 =$ **2**: 172.16.0.0, 172.16.2.0, 172.16.4.0 …

#	Subnet	First host	Last host	Broadcast	Hosts
1	172.16.0.0/23	172.16.0.1	172.16.1.254	172.16.1.255	510
2	172.16.2.0/23	172.16.2.1	172.16.3.254	172.16.3.255	510
3	172.16.4.0/23	172.16.4.1	172.16.5.254	172.16.5.255	510
4	172.16.6.0/23	172.16.6.1	172.16.7.254	172.16.7.255	510
5	172.16.8.0/23	172.16.8.1	172.16.9.254	172.16.9.255	510
…					

Note how the host range crosses an octet boundary: 172.16.0.1 → 172.16.1.254 is one single subnet.

10.7 Example E – class A: 10.0.0.0/8 into 16 subnets

$16 = 2^4$ → /12 → mask 255.240.0.0, interesting octet the 2nd, block 16: 10.0.0.0, 10.16.0.0, 10.32.0.0 … 10.240.0.0. Each has $2^{20} - 2 = 1{,}048{,}574$ hosts.

#	Subnet	First host	Last host	Broadcast	Hosts
1	10.0.0.0/12	10.0.0.1	10.15.255.254	10.15.255.255	1,048,574
2	10.16.0.0/12	10.16.0.1	10.31.255.254	10.31.255.255	1,048,574
3	10.32.0.0/12	10.32.0.1	10.47.255.254	10.47.255.255	1,048,574
4	10.48.0.0/12	10.48.0.1	10.63.255.254	10.63.255.255	1,048,574
5	10.64.0.0/12	10.64.0.1	10.79.255.254	10.79.255.255	1,048,574
…					

Subnet zero and the all-ones subnet
Old textbooks (RFC 950) said the first subnet (all borrowed bits 0) and the last one (all 1) must not be used, giving $2^s - 2$ subnets. Since RFC 1878 (1995) and on all modern equipment (Cisco `ip subnet-zero` is on by default), **all 2^s subnets are usable**. Use 2^s unless an exam explicitly states otherwise.

11. Fast subnetting: the “magic number” method

Once binary is understood, this mental method answers most questions in seconds, without writing 32 bits.

1. Find the **interesting octet**: the octet where the mask is neither 255 nor 0.
2. **Magic number** (block size) = 256 − mask value in that octet.
3. Network: the largest **multiple of the magic number** that is ≤ the address value in the interesting octet. Octets to the right → 0.
4. Broadcast: next network − 1 in the interesting octet; octets to the right → 255.

Example: 200.15.7.99/29

- /29 = 255.255.255.248 → interesting octet = 4th, magic number = 256 − 248 = **8**.
- Multiples of 8: ... 88, **96**, 104 ... → 99 falls in the block 96–103.
- Network **200.15.7.96**, broadcast **200.15.7.103**, hosts .97 – .102 (6 hosts).

Example: 172.16.45.200/20 (again)

- 255.255.240.0 → 3rd octet, magic number 16 → multiples 0, 16, **32**, 48 → 45 is in 32–47.
- Network 172.16.32.0, broadcast 172.16.47.255. Same as the binary method, much faster.

Are two hosts in the same subnet?

192.168.1.62/26 and 192.168.1.65/26: block 64 → .62 is in 0–63, .65 is in 64–127 → **different subnets**; they need a router to communicate even though their addresses are “close”.

Quick reference: prefix ↔ magic number

4th octet	/25: 128	/26: 64	/27: 32	/28: 16	/29: 8	/30: 4	/31: 2	/32: 1
3rd octet	/17: 128	/18: 64	/19: 32	/20: 16	/21: 8	/22: 4	/23: 2	/24: 1
2nd octet	/9: 128	/10: 64	/11: 32	/12: 16	/13: 8	/14: 4	/15: 2	/16: 1

12. VLSM – Variable Length Subnet Masking

With FLSM, all subnets are the size of the largest need: a link between two routers (2 hosts) would consume a /26 of 62 addresses – 60 wasted. **VLSM** allows **a different mask for each subnet**, sized to its real need. It requires classless routing protocols (OSPF, EIGRP, RIPv2, BGP, static routes) – all modern ones.

12.1 The method

1. List all needs (hosts per network, plus point-to-point links).
2. **Sort them from largest to smallest** – essential to keep blocks aligned and avoid overlaps.
3. For each need, choose the smallest prefix with $2^h - 2 \geq$ hosts (remember to count the router interface/gateway as a host).
4. Allocate the first subnet at the start of the block; each next subnet starts right after the previous broadcast address.
5. Check that a subnet always starts on a multiple of its own block size.

12.2 Worked example

Address block **192.168.10.0/24**. Needs: LAN A 100 hosts, LAN B 50, LAN C 25, LAN D 10, three WAN links between routers (2 hosts each).

Network	Hosts needed	Prefix (usable)	Subnet	Mask	Host range	Broadcast
LAN A	100	/25 (126)	192.168.10.0/25	255.255.255.128	192.168.10.1 – 192.168.10.126	192.168.10.127
LAN B	50	/26 (62)	192.168.10.128/26	255.255.255.192	192.168.10.129 – 192.168.10.190	192.168.10.191
LAN C	25	/27 (30)	192.168.10.192/27	255.255.255.224	192.168.10.193 – 192.168.10.222	192.168.10.223
LAN D	10	/28 (14)	192.168.10.224/28	255.255.255.240	192.168.10.225 – 192.168.10.238	192.168.10.239
WAN 1	2	/30 (2)	192.168.10.240/30	255.255.255.252	192.168.10.241 – 192.168.10.242	192.168.10.243
WAN 2	2	/30 (2)	192.168.10.244/30	255.255.255.252	192.168.10.245 – 192.168.10.246	192.168.10.247
WAN 3	2	/30 (2)	192.168.10.248/30	255.255.255.252	192.168.10.249 – 192.168.10.250	192.168.10.251

Detail of choices: 100 → $2^7 - 2 = 126$ → /25; 50 → 62 → /26; 25 → 30 → /27; 10 → 14 → /28; 2 → 2 → /30. Remaining free: 192.168.10.252/30 for future use. In FLSM with /25 masks we could have created only 2 networks; VLSM fits 7 networks in the same /24.

Overlap: the classic VLSM mistake

If you allocate small subnets first (e.g. 192.168.10.0/30), then try to place a /25 at 192.168.10.0, the two overlap. A /25 can only start at .0 or .128, a /26 at .0/.64/.128/.192, etc. Sorting from largest to smallest guarantees correct alignment.

13. Supernetting and route summarization

13.1 Principle

Supernetting is the opposite of subnetting: several contiguous networks are combined into one larger network by **shortening the prefix** (moving bits from the network part back to the host part). Used for:

- **Route summarization (aggregation)**: a router advertises one route instead of many → smaller routing tables, less CPU and memory, faster convergence, and instabilities of a small network are hidden from the rest.
- **CIDR allocation**: an ISP gives a customer who needs 1,000 addresses a **/22** (1,024 addresses) instead of four separate class C networks.

13.2 Method: find the common bits

1. Write the networks in binary (only the octet that differs is needed).
2. Count the **left-most bits common** to all of them → that is the summary prefix.
3. Summary address = common bits followed by zeros.
4. Verify that the summary does not include networks that are not yours.

Example 1 – 192.168.0.0/24 to 192.168.3.0/24

```
192.168.0.0   11000000.10101000.00000000.00000000
192.168.1.0   11000000.10101000.00000001.00000000
192.168.2.0   11000000.10101000.00000010.00000000
192.168.3.0   11000000.10101000.00000011.00000000
              |<-------- 22 common bits -------->|
```

Summary: **192.168.0.0/22** (mask 255.255.252.0), covering exactly 192.168.0.0 – 192.168.3.255.

Example 2 – 172.16.12.0/24 to 172.16.15.0/24

```
12 = 00001100    13 = 00001101    14 = 00001110    15 = 00001111
```

16 + 6 = 22 common bits → **172.16.12.0/22**.

Example 3 – 10.1.64.0/24 to 10.1.127.0/24 (64 networks)

64 = **01**000000, 127 = **01**111111 → 16 + 2 = 18 common bits → **10.1.64.0/18**. One route replaces 64.

Example 4 – two /25 into a /24

198.51.100.0/25 + 198.51.100.128/25 → 198.51.100.0/24.

13.3 Rules and traps

- The number of networks summarized exactly is always a **power of two** (2, 4, 8, 16…).

- The first network must be a **multiple of that number** in the interesting octet (4 networks → start at 0, 4, 8, 12…; 8 networks → 0, 8, 16…).

Non-aligned networks: 192.168.1.0/24 to 192.168.4.0/24
1 = 00000**001**, 2 = 00000**010**, 3 = 00000**011**, 4 = 00000**100**: only 21 bits in common → 192.168.0.0/21, which also covers 192.168.0.0, .5.0, .6.0 and .7.0 – networks that may belong to someone else (**over-summarization**, causing traffic to be black-holed). The exact minimum summary is three routes: **192.168.1.0/24**, **192.168.2.0/23**, **192.168.4.0/24**.

Quick method without binary

Count the networks (N), round up to a power of two 2^k, the summary prefix is *old prefix* − *k*; check that the first network is a multiple of 2^k. Example: 8 networks 10.0.8.0/24 … 10.0.15.0/24 → 2^3 → /24 − 3 = /21; 8 is a multiple of 8 ✓ → **10.0.8.0/21**.

14. Wildcard masks

A **wildcard mask** is the inverse of the subnet mask: **0 = this bit must match, 1 = any value**. Used in Cisco ACLs and in OSPF/EIGRP `network` statements.

Calculation: **255.255.255.255 − subnet mask**.

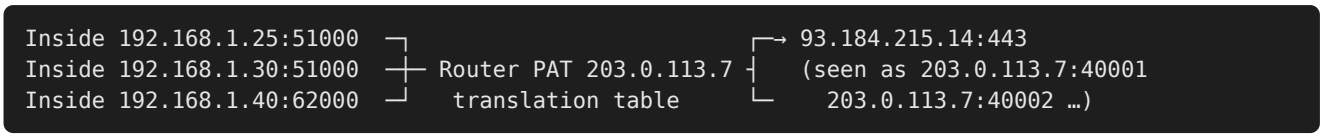
Prefix	Subnet mask	Wildcard
/24	255.255.255.0	0.0.0.255
/27	255.255.255.224	0.0.0.31
/20	255.255.240.0	0.0.15.255
/21	255.255.248.0	0.0.7.255
/32 (one host)	255.255.255.255	0.0.0.0 (keyword `host`)
/0 (everything)	0.0.0.0	255.255.255.255 (keyword `any`)

```
access-list 10 permit 192.168.10.64 0.0.0.63        ! the /26 192.168.10.64 – .127
router ospf 1
 network 10.1.64.0 0.0.63.255 area 0                  ! the /18 10.1.64.0
```

15. NAT and PAT

NAT (Network Address Translation, RFC 3022) lets hosts with private addresses reach the Internet: the router replaces the private source address with a public one and remembers the mapping to translate replies back.

Type	Principle	Use
Static NAT	1 private ↔ 1 public, permanent	Publishing an internal server
Dynamic NAT	Private addresses take a free public address from a pool	Rare today
PAT / NAT overload	Many private addresses share **one** public address, distinguished by the **source port**	Every home box and most companies
Port forwarding	Public IP:port → private IP:port	Hosting a game/web server at home
CGNAT	The ISP itself does NAT (100.64.0.0/10 between ISP and customer)	Mobile networks, ISPs short of IPv4



Terminology (Cisco): *inside local* = private address of the host, *inside global* = public address representing it, *outside global* = address of the remote server. Drawbacks of NAT: breaks end-to-end connectivity, complicates incoming connections, VoIP and some VPN protocols – IPv6 removes the need for it.

To know your public address: `curl ifconfig.me` (Linux) or `curl.exe ifconfig.me` (Windows) – compare it with `ipconfig` / `ip a` which shows the private address.

16. IPv6 addressing

16.1 Format

IPv6 (RFC 8200) uses **128-bit** addresses: $2^{128} \approx 3.4 \times 10^{38}$ addresses. They are written as **8 groups of 4 hexadecimal digits** (16 bits each, called *hextets*) separated by colons.

```
2001:0db8:0000:0000:0000:ff00:0042:8329
```

Abbreviation rules (RFC 5952)

1. **Leading zeros** of each group may be removed: `0db8 → db8`, `0042 → 42`, `0000 → 0`.
2. **One** sequence of consecutive all-zero groups may be replaced by `::` – only once per address (otherwise the length would be ambiguous). Replace the longest run; if two runs are equal, the left-most one.
3. Use lowercase letters.

```
2001:0db8:0000:0000:0000:ff00:0042:8329  →  2001:db8::ff00:42:8329
2001:0db8:0000:0000:0001:0000:0000:0001  →  2001:db8::1:0:0:1        (left-most run compressed)
0000:0000:0000:0000:0000:0000:0000:0001  →  ::1                     (loopback)
fe80::1   expanded  →  fe80:0000:0000:0000:0000:0000:0000:0001
```

To expand: count the groups present, the `::` stands for as many 0000 groups as needed to reach 8.

16.2 Structure of a global unicast address

```
|<--- 48 bits --->|<- 16 bits ->|<---------- 64 bits ---------->|
  Global routing      Subnet ID          Interface ID
  prefix (from ISP)   (your subnets)     (identifies the host)
  2001:db8:abcd    :    0012       :  3e52:82ff:fe1a:4f9e
```

A LAN in IPv6 is **always a /64**. A site usually receives a **/48** (or /56 for homes), leaving 16 bits (or 8) for subnets: 65,536 subnets of 2^{64} hosts each. Subnetting IPv6 is therefore simple: you just increment the subnet ID in hex: 2001:db8:abcd:**0000**::/64, 2001:db8:abcd:**0001**::/64 … 2001:db8:abcd:**ffff**::/64.

16.3 Address types

Prefix	Type	IPv4 equivalent
2000::/3	**Global unicast** (public, routable)	Public addresses
fe80::/10	**Link-local** – automatically present on every IPv6 interface, used for neighbour discovery and routing protocols	169.254.0.0/16 (but always present)
fc00::/7 (in practice fd00::/8)	**Unique local** (ULA) – internal, not routed on the Internet	RFC 1918 private
ff00::/8	**Multicast** (ff02::1 all nodes, ff02::2 all routers)	224.0.0.0/4
::1/128	Loopback	127.0.0.1
::/128	Unspecified	0.0.0.0
::/0	Default route	0.0.0.0/0
2001:db8::/32	Documentation	192.0.2.0/24 etc.
::ffff:0:0/96	IPv4-mapped (::ffff:192.0.2.1)	—

There is **no broadcast** in IPv6: it is replaced by multicast. **Anycast** addresses are normal unicast addresses assigned to several devices.

16.4 How hosts get an IPv6 address

- **SLAAC** (Stateless Address Autoconfiguration): the router sends a *Router Advertisement* with the /64 prefix; the host builds its interface ID itself (random, for privacy – RFC 8981 – or with EUI-64).
- **DHCPv6**: stateful (server assigns addresses) or stateless (only DNS info, address by SLAAC).
- **Static** configuration.

EUI-64 example: MAC 3c:52:82:1a:4f:9e

1. Split the MAC in two halves: 3c5282 | 1a4f9e.
2. Insert `fffe` in the middle: 3c5282**fffe**1a4f9e.
3. Invert the 7th bit (U/L bit) of the first byte: 3c = 0011 11**0**0 → 0011 11**1**0 = **3e**.
4. Interface ID: **3e52:82ff:fe1a:4f9e** → link-local fe80::3e52:82ff:fe1a:4f9e.

17. Reference tables

17.1 Complete CIDR table (/8 to /32)

Prefix	Subnet mask	Wildcard	Addresses	Usable hosts
/8	255.0.0.0	0.255.255.255	16,777,216	16,777,214
/9	255.128.0.0	0.127.255.255	8,388,608	8,388,606
/10	255.192.0.0	0.63.255.255	4,194,304	4,194,302
/11	255.224.0.0	0.31.255.255	2,097,152	2,097,150
/12	255.240.0.0	0.15.255.255	1,048,576	1,048,574
/13	255.248.0.0	0.7.255.255	524,288	524,286
/14	255.252.0.0	0.3.255.255	262,144	262,142
/15	255.254.0.0	0.1.255.255	131,072	131,070
/16	255.255.0.0	0.0.255.255	65,536	65,534
/17	255.255.128.0	0.0.127.255	32,768	32,766
/18	255.255.192.0	0.0.63.255	16,384	16,382
/19	255.255.224.0	0.0.31.255	8,192	8,190
/20	255.255.240.0	0.0.15.255	4,096	4,094
/21	255.255.248.0	0.0.7.255	2,048	2,046
/22	255.255.252.0	0.0.3.255	1,024	1,022
/23	255.255.254.0	0.0.1.255	512	510
/24	255.255.255.0	0.0.0.255	256	254
/25	255.255.255.128	0.0.0.127	128	126
/26	255.255.255.192	0.0.0.63	64	62
/27	255.255.255.224	0.0.0.31	32	30
/28	255.255.255.240	0.0.0.15	16	14
/29	255.255.255.248	0.0.0.7	8	6
/30	255.255.255.252	0.0.0.3	4	2
/31	255.255.255.254	0.0.0.1	2	2 (point-to-point, RFC 3021)
/32	255.255.255.255	0.0.0.0	1	1 (single host)

17.2 Choosing a prefix from a number of hosts

Hosts needed	Host bits	Prefix	Usable
2	2	/30	2
3 – 6	3	/29	6
7 – 14	4	/28	14
15 – 30	5	/27	30
31 – 62	6	/26	62
63 – 126	7	/25	126
127 – 254	8	/24	254
255 – 510	9	/23	510
511 – 1,022	10	/22	1,022
1,023 – 2,046	11	/21	2,046
2,047 – 4,094	12	/20	4,094

17.3 Common mistakes checklist

- Forgetting the −2 (network and broadcast) when counting hosts.
- Forgetting that the router (gateway) also needs an address in each subnet.
- Thinking 172.32.x.x is private (only 172.16–172.31 is).
- Judging an address invalid because it ends in .0 or .255 without computing the subnet.
- Using a non-contiguous mask (255.255.255.64).
- Not sorting needs from largest to smallest in VLSM → overlapping subnets.
- Summarizing a range that is not a power of two or not aligned → over-summarization.
- Using `::` twice in an IPv6 address.

18. Exercises

Part A – Binary

1. Convert to binary: 200, 99, 7, 255.
2. Convert to decimal: 10110111, 01111111, 11100000.
3. Convert 0xC0 and 0xFE to decimal and binary.

Part B – Address types

4. For each address, give the class and say whether it is public, private or special: 10.20.30.40 · 172.32.1.1 · 172.31.0.9 · 192.168.300.1 · 169.254.10.5 · 100.70.1.1 · 8.8.8.8 · 224.0.0.5 · 127.0.0.53.

Part C – Network calculations

5. 192.168.50.130/27: mask, network, broadcast, host range, number of hosts.
6. 172.20.99.14/19: same questions.
7. 10.200.17.5/10: same questions.
8. Is 192.168.1.255/23 a valid host address? And 192.168.1.0/23?
9. Are 192.168.1.62/26 and 192.168.1.65/26 in the same subnet?

Part D – Subnetting and VLSM

10. Divide 192.168.100.0/24 into 8 equal subnets. Give the prefix and the list of networks.
11. What is the smallest prefix for: 300 hosts? 2 hosts? 1,000 hosts? 14 hosts?
12. VLSM: with 10.10.0.0/24, address a Sales LAN (60 hosts), a Tech LAN (28), an Admin LAN (12) and a router-to-router link.

Part E – Supernetting, wildcards, IPv6

13. Summarize 172.16.32.0/24, 172.16.33.0/24, 172.16.34.0/24, 172.16.35.0/24.
14. Summarize 10.0.8.0/24 through 10.0.15.0/24.
15. Give the wildcard mask of /27 and of 255.255.248.0.
16. Compress 2001:0db8:0000:0000:0001:0000:0000:0001. Expand fe80::1.

Answers

1. 200 = 11001000 (128+64+8) · 99 = 01100011 (64+32+2+1) · 7 = 00000111 · 255 = 11111111.

2. 10110111 = 128+32+16+4+2+1 = 183 · 01111111 = 127 · 11100000 = 224.

3. 0xC0 = 12×16 = 192 = 11000000 · 0xFE = 15×16+14 = 254 = 11111110.

4. 10.20.30.40: class A, **private** · 172.32.1.1: class B, **public** (outside 172.16–31) · 172.31.0.9: class B, private · 192.168.300.1: **invalid** (300 > 255) · 169.254.10.5: class B range, **link-local/APIPA** · 100.70.1.1: class A range, **shared CGNAT space** 100.64.0.0/10 · 8.8.8.8: class A, public · 224.0.0.5: class D, **multicast** (OSPF) · 127.0.0.53: **loopback** (used by systemd-resolved).

5. /27 = 255.255.255.224, block 32; 130 is in 128–159 → network 192.168.50.128, broadcast 192.168.50.159, hosts .129 – .158, 30 hosts.

6. /19 = 255.255.224.0, 3rd octet, block 32; 99 is in 96–127 → network 172.20.96.0, broadcast 172.20.127.255, hosts 172.20.96.1 – 172.20.127.254, $2^{13} - 2 = 8{,}190$ hosts.

7. /10 = 255.192.0.0, 2nd octet, block 64; 200 is in 192–255 → network 10.192.0.0, broadcast 10.255.255.255, hosts 10.192.0.1 – 10.255.255.254, $2^{22} - 2 = 4{,}194{,}302$ hosts.

8. /23 → block 2 in the 3rd octet → subnet 192.168.0.0 – 192.168.1.255. 192.168.1.255 is the **broadcast** → not a valid host. 192.168.1.0 is in the middle of the range → **valid host**.

9. No: block 64 → .62 belongs to 192.168.1.0/26, .65 to 192.168.1.64/26.

10. $8 = 2^3$ → /27 (255.255.255.224), block 32: .0, .32, .64, .96, .128, .160, .192, .224 (see table in 10.5).

11. 300 → /23 (510) · 2 → /30 (or /31 on a point-to-point link) · 1,000 → /22 (1,022) · 14 → /28 (exactly 14).

12. Sorted from largest to smallest:

Network	Hosts needed	Prefix (usable)	Subnet	Mask	Host range	Broadcast
Sales	60	/26 (62)	10.10.0.0/26	255.255.255.192	10.10.0.1 – 10.10.0.62	10.10.0.63
Tech	28	/27 (30)	10.10.0.64/27	255.255.255.224	10.10.0.65 – 10.10.0.94	10.10.0.95
Admin	12	/28 (14)	10.10.0.96/28	255.255.255.240	10.10.0.97 – 10.10.0.110	10.10.0.111
Link R1–R2	2	/30 (2)	10.10.0.112/30	255.255.255.252	10.10.0.113 – 10.10.0.114	10.10.0.115

13. 32 = 00100**00** … 35 = 00100**11** → 22 common bits → **172.16.32.0/22**.

14. 8 networks, first = 8 (multiple of 8) → /24 − 3 = **10.0.8.0/21**.

15. /27 → 0.0.0.31 · 255.255.248.0 → 0.0.7.255.

16. 2001:db8::1:0:0:1 (two runs of two zero groups: the left-most is compressed, and :: is used only once) · fe80:0000:0000:0000:0000:0000:0000:0001.

Check your work with tools

Linux: `ipcalc 172.16.45.200/20` or `sipcalc`. Python: `python3 -c "import ipaddress as i; n=i.ip_interface('172.16.45.200/20').network; print(n, n.broadcast_address, n.num_addresses-2)"`. Windows PowerShell has no built-in calculator, but Python or online subnet calculators work. Use them to **verify**, never to replace understanding – exams and real troubleshooting require mental calculation.